



Configure CORS

[Cross-Origin Resource Sharing \(CORS\)](#) [↗] is a standardized method that prevents domain X from accessing the resources of domain Y. It does so by using special headers in HTTP responses from domain Y, that allow your browser to verify that domain Y permits domain X to access these resources.

While CORS can help protect your data from malicious websites, CORS is also used to interact with objects in your bucket and configure policies on your bucket.

CORS is used when you interact with a bucket from a web browser, and you have two options:

Set a bucket to public: This option makes your bucket accessible on the Internet as read-only, which means anyone can request and load objects from your bucket in their browser or anywhere else. This option is ideal if your bucket contains images used in a public blog.

Presigned URLs: Allows anyone with access to the unique URL to perform specific actions on your bucket.

Prerequisites

Before you configure CORS, you must have:

- An R2 bucket with at least one object. If you need to create a bucket, refer to [Create a public bucket](#).
- A domain you can use to access the object. This can also be a localhost.
- (Optional) Access keys. An access key is only required when creating a presigned URL.

Use CORS with a public bucket

[To use CORS with a public bucket](#), ensure your bucket is set to allow public access.

Next, [add a CORS policy](#) to your bucket to allow the file to be shared.

Use CORS with a presigned URL

Presigned URLs are an S3 concept that contain a special signature that encodes details of an S3 action, such as `GetObject` or `PutObject`. Presigned URLs are only used for authentication, which means they are generally safe to distribute publicly without revealing any secrets.

Create a presigned URL

You will need a pair of S3-compatible credentials to use when you generate the presigned URL.

The example below shows how to generate a presigned PutObject URL using the [@aws-sdk/client-s3 ↗](#) package for JavaScript.

JavaScript

```
import { PutObjectCommand, S3Client } from
"@aws-sdk/client-s3";
import { getSignedUrl } from "@aws-sdk/s3-
request-presigner";
const S3 = new S3Client({
  endpoint:
"https://<account_id>.r2.cloudflarestorage.com",
  credentials: {
    accessKeyId: "<access_key_id>",
    secretAccessKey: "<access_key_secret>",
  },
  region: "auto",
});
const url = await getSignedUrl(
  S3,
  new PutObjectCommand({
    Bucket: bucket,
    Key: object,
  }),
  {
    expiresIn: 60 * 60 * 24 * 7, // 7d
  },
);
console.log(url);
```

Test the presigned URL

Test the presigned URL by uploading an object using cURL. The example below would upload the 123 text to R2 with a Content-Type of text/plain.

```
curl --request PUT <URL> --header  
"Content-Type: text/plain" --data "123"
```

Add CORS policies from the dashboard

1. In the Cloudflare dashboard, go to the **R2 object storage** page.

Go to **Overview** [↗](#)

2. Locate and select your bucket from the list.
3. Select **Settings**.

4. Under **CORS Policy**, select **Add CORS policy**.
5. From the **JSON** tab, manually enter or copy and paste your policy into the text box.
6. When you are done, select **Save**.

Your policy displays on the **Settings** page for your bucket.

Response headers

The following fields in an R2 CORS policy map to HTTP response headers. These response headers are only returned when the incoming HTTP request is a valid CORS request.

Field Name	Description	Example
AllowedOrigins	Specifies the value for the	If a website
Access-Control-Allow-Origin	Access-Control-Allow-Origin	www.test.com
Access-Control-Allow-Methods	Access-Control-Allow-Methods	access resc
Access-Control-Allow-Headers	Access-Control-Allow-Headers	fonts, scrip

Field Name	Description	Example
	Origin header	custom domain
	R2 sets when	static.example.com
	requesting	you would
	objects in a	https://www.example.com
	bucket from a	as an AllowedOrigin
	browser.	
AllowedMethods	Specifies the value for the Access-Control-Allow-Methods header	GET, POST, PUT
	R2 sets when	
	requesting	
	objects in a	
	bucket from a	
	browser.	
AllowedHeaders	Specifies the value for the Access-Control-Allow-Headers header	x-requested-with
	Agent	

Field Name	Description	Example
	Control-Allow-Headers header R2 sets when requesting objects in this bucket from a browser.Cross-origin requests that include custom headers (e.g. x-user-id) should specify these headers as AllowedHeaders.	
ExposeHeaders	Specifies the headers that can be exposed back, and accessed by, the	Content-En cache-stat

Field Name	Description	Example
	JavaScript making the cross-origin request. If you need to access headers beyond the safelisted response headers ↗ , such as Content-Encoding or cf-cache-status, you must specify it here.	
MaxAgeSeconds	Specifies the amount of time (in seconds) browsers are allowed to cache	3600

Field Name	Description	Example
	CORS preflight responses. Browsers may limit this to 2 hours or less, even if the maximum value (86400) is specified.	

Example

This example shows a CORS policy added for a bucket that contains the `Roboto-Light.ttf` object, which is a font file.

The `AllowedOrigins` specify the web server being used, and `localhost:3000` is the hostname where the web server is running. The `AllowedMethods` specify that

only GET requests are allowed and can read objects in your bucket.

```
[  
  {  
    "AllowedOrigins":  
    ["http://localhost:3000"],  
    "AllowedMethods": ["GET"]  
  }  
]
```

In general, a good strategy for making sure you have set the correct CORS rules is to look at the network request that is being blocked by your browser.

- Make sure the rule's `AllowedOrigins` includes the origin where the request is being made from. (like `http://localhost:3000` or `https://yourdomain.com`)
- Make sure the rule's `AllowedMethods` includes the blocked request's method.

- Make sure the rule's `AllowedHeaders` includes the blocked request's headers.

Also note that CORS rule propagation can, in rare cases, take up to 30 seconds.

Common Issues

- Only a cross-origin request will include CORS response headers.
 - A cross-origin request is identified by the presence of an `Origin` HTTP request header, with the value of the `Origin` representing a valid, allowed origin as defined by the `AllowedOrigins` field of your CORS policy.
 - A request without an `Origin` HTTP request header will *not* return any CORS response headers. Origin values must match exactly.
- The value(s) for `AllowedOrigins` in your CORS policy must be a valid [HTTP Origin header](https://developers.cloudflare.com/r2/buckets/cors/)

[value ↗](#). A valid Origin header does *not* include a path component and must only be comprised of a scheme://host[:port] (where port is optional).

- Valid AllowedOrigins value:
`https://static.example.com` - includes the scheme and host. A port is optional and implied by the scheme.
- Invalid AllowedOrigins value:
`https://static.example.com/` or
`https://static.example.com/fonts/Calibri.woff2` - incorrectly includes the path component.
- If you need to access specific header values via JavaScript on the origin page, such as when using a video player, ensure you set `Access-Control-Expose-Headers` correctly and include the headers your JavaScript needs access to, such as `Content-Length`.

Was this helpful?



[Previous](#)[← Public buckets](#)[Next](#)[Bucket locks →](#)

Last updated: Sep 3, 2025

Resources

[API](#)[New to Cloudflare?](#)[Directory](#)[Sponsorships](#)[Open Source](#)

Company

[cloudflare.com](#)[Our team](#)[Careers](#)

Support

[Help Center](#)[System Status](#)[Compliance](#)[GDPR](#)

Tools

[Cloudflare Radar](#)[Speed Test](#)[Is BGP Safe Yet?](#)[RPKI Toolkit](#)[Certificate Transparency](#)

Community



© 2025 Cloudflare, Inc. • [Privacy Policy](#) • [Terms of Use](#)

• [Report Security Issues](#) • [Trademark](#) •  [Cookie Preferences](#)